AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 8.3: Vector Storage, Retrieval, and GraphRAG

Zhigang Feng

Spring 2026

Topic 8.3 Agenda

1. **Vector Storage and Retrieval** — vector DBs, ANN search
2. **Beyond Dense Retrieval** — hybrid, GraphRAG

Arc: T1 Prompting wall → T2 Chunking & embeddings → **T3 Vector & graph retrieval** *→ T4a Augmented generation → T4b Demo, failures, agentic bridge*

Vector Storage and Retrieval

Why a Vector Database?

- Now we have millions of chunks, each with a 1024-dim embedding. **Where do they live?**
- **The naive option:** a flat numpy array on disk. Search by computing cosine similarity to every vector and sorting.
  - Works fine for $10{,}000$ vectors.
  - At $1{,}000{,}000$ vectors a single query takes seconds.
  - At $10{,}000{,}000$ it is impractical.
  - No metadata filters. No incremental updates. No persistence.
- **A vector database** solves all four:
  1. *Scale:* approximate nearest-neighbor (ANN) algorithms search millions of vectors in milliseconds.
  2. *Metadata filters:* restrict search to "date $\geq$ 2020 AND doc_type $=$ minutes."
  3. *Incremental updates:* add new chunks as new documents arrive, without rebuilding the index.
  4. *Persistence:* the index lives on disk and survives process restarts.

Vector DB Landscape — Scale vs Hosting Cost

For an economist starting out:

- **Chroma** or **FAISS** for laptop / cluster work — bottom-left of the chart. Free, local, simple.

Similarity Search 101

- Given a query vector q and a database of vectors $\{d_1, d_2, \ldots, d_N\}$, find the K closest d_i to q.
- **Three common similarity functions:**
  - **Cosine similarity:** $\cos(\theta) = \dfrac{q \cdot d_i}{\|q\| \, \|d_i\|}$
  - **Dot product:** $q \cdot d_i$
  - **(Negative) Euclidean distance:** $-\|q - d_i\|$
- **Why cosine is the default:** Cosine is invariant to vector magnitude. Modern embedders are typically L2-normalized at the end of the forward pass, in which case cosine = dot product and both reduce to: *higher dot product* $\Rightarrow$ *more similar*.
- **Geometric picture:** Each chunk is an arrow from the origin. The query is another arrow. Cosine similarity measures the angle between them. Two vectors pointing in the same direction (small angle) are deemed similar regardless of length.

Approximate Nearest Neighbor (ANN)

- **Exact search** computes similarity to *every* vector in the database. Cost: $O(N \cdot d)$ per query.
- For $N = 10^7$ and $d = 1024$, that's $\sim 10^{10}$ floating-point operations per query — seconds of latency.
- **Approximate Nearest Neighbor:** Sacrifice a tiny bit of accuracy ($\sim$1% recall loss) for orders-of-magnitude speedup.
- **Two dominant ANN families:**
  - **HNSW** (Hierarchical Navigable Small World): build a multi-layer graph of vectors; search by greedy walks on the graph. Default in Qdrant, Weaviate, pgvector.
  - **IVF** (Inverted File): cluster vectors into Voronoi cells, search only the closest cells. Default in FAISS for very large indexes.
- **In practice** you won't choose the algorithm by hand — you'll pick a vector DB and accept its default. But knowing the names helps when you read the docs.

Hybrid Retrieval: Dense + Sparse

- **The dense-only failure case:** A user asks "find every speech mentioning *Section 13(3)*." This is a precise legal-citation query.
  - Dense embeddings cluster similar *meanings*, not similar *strings*.
  - "Section 13(3)" may not even be in the embedder's vocabulary.
  - Result: top-10 neighbors are about the discount window in general — not the specific section.
- **The fix:** also run a classical sparse retriever in parallel.
  - **BM25**: a TF-IDF-based ranking function that scores documents by exact term matches with the query.
  - Catches everything dense retrieval misses on rare strings, acronyms, statute citations, ticker symbols, identifiers.
- **Combine the two via Reciprocal Rank Fusion (RRF):**

$$\text{score}(d) = \sum_{r \in \{\text{dense, sparse}\}} \frac{1}{k + \text{rank}_r(d)} \quad \text{(typically } k = 60\text{)}$$

- Hybrid retrieval is the **production default in 2025**. Pure dense is rarely competitive.

Re-Ranking with Cross-Encoders

- **Two-stage retrieval:**
  1. **First pass (bi-encoder):** fast, embeds query and documents *independently*, returns top-50 candidates by cosine similarity.
  2. **Second pass (cross-encoder re-ranker):** slow, takes the query *and* a candidate document *together* as input, outputs a relevance score. Returns the top-5.

- **Why this works.** A cross-encoder reads the query and document jointly — so it can attend to which parts of the document actually answer the query. Bi-encoders cannot do this. The cost: cross-encoders are $\sim$100$\times$ slower per pair, so we only use them on the top-50 short-list.

- **Popular re-rankers:**
  - Cohere `Rerank` (paid API)
  - `bge-reranker-large` (free, local)
  - `ms-marco-MiniLM-L-12-v2` (free, smaller)

- **Why re-ranking matters in practice.** The top-1 hit from a bi-encoder is often outranked by something the re-ranker catches. Without re-ranking you ask the LLM the wrong question.

Metadata Filters

- Pure semantic search is insufficient when the query has *structural* constraints.
- **Real example:** *"Find all FOMC discussions of unemployment from 2020 onwards, excluding press conferences."*
  - "Discussions of unemployment" $\Rightarrow$ semantic search.
  - "From 2020 onwards" $\Rightarrow$ filter on `date` $\geq$ `2020-01-01`.
  - "Excluding press conferences" $\Rightarrow$ filter on `doc_type` $\neq$ `press_conference`.
- Modern vector DBs let you combine vector similarity with SQL-style metadata filters in a single query.
- **This is exactly why preserving rich metadata at chunking time matters** (Section IV gotcha). If your chunks don't carry `date`, `doc_type`, `speaker`, etc., you cannot filter on them later.
- **Rule of thumb:** Add *more* metadata than you think you need. It is cheap at index time and impossible to reconstruct after the fact.

Code: Build a Chroma Index and Query It

```python
import chromadb
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("BAAI/bge-large-en-v1.5")

# 1. Persistent Chroma client (data lives on disk)
client = chromadb.PersistentClient(path="./fomc_index")
coll = client.get_or_create_collection("fomc")

# 2. Add chunks with metadata
coll.add(
    ids=[f"chunk_{i}" for i in range(len(chunks))],
    documents=chunks,
    embeddings=model.encode(chunks, normalize_embeddings=True).tolist(),
    metadatas=[{"meeting": "2024-06", "doc_type": "minutes"} for _ in
        chunks],
)
```

Beyond Dense Retrieval

Where Vanilla Vector RAG Starts to Break

- Dense retrieval is a **local similarity machine**. It finds chunks whose wording or meaning is close to the query.
- That is enough for many lookup questions, but it misses three important structures:
  1. **Relationships between objects.** Two chunks may matter because their entities are linked, even if the wording differs.
  2. **Global structure.** Nearest-neighbor search has no notion of discourse communities, themes, or argument clusters across an entire corpus.
  3. **Coverage.** If the same fact appears in many chunks, top-K retrieval often returns duplicates instead of a representative overview.
- **Economist examples:**
  - "Which Senators most often connect inflation to housing supply constraints?"
  - "What are the main camps in recent papers on LLMs and education?"
- Once the question is **multi-hop** or **corpus-level**, plain top-K chunk retrieval is often the wrong abstraction.

Concrete Break Point: Where Vanilla Fails (and Why GraphRAG Follows)

Question: *"Which Phillips-curve papers use survey expectations and are cited by post-2020 FOMC speeches?"*

Vanilla vector RAG

- Embeds the query as one vector.
- Returns 5 chunks that *sound like* "Phillips curve + survey expectations."
- Has **no notion of citation edges** between NBER papers and FOMC speeches.
- Best-case output: a chunk that *discusses* both topics, even if no actual citation link exists.
- Failure mode: confident, fluent, but the cited link is fictional.

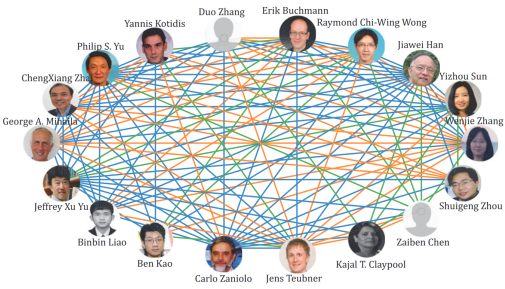
GraphRAG

- Walks the citation graph: NBER paper $\rightarrow$ `cited_by` edge $\rightarrow$ FOMC speech node.
- Filters by node attribute: `date` $\geq$ 2020.
- Filters by paper attribute: `uses_data: SurveyOfProfessionalForecasters` or similar.
- Returns the actual subgraph; only then runs generation on the supporting chunks.
- Failure mode: degrades to "no path found," not invention.

Rule: reach for GraphRAG when the question is *about a relationship*, not just *about a topic*. Top-K similarity cannot manufacture an edge that the index does not represent.

Knowledge Graphs: Adding Structure to Retrieval

- A **knowledge graph (KG)** stores facts as **nodes + edges** rather than isolated chunks.
- Canonical representation: `<head, relation, tail>`, e.g. `<Powell, discusses, inflation expectations>`.
- In graph-based RAG, the index can mix **entity nodes**, **chunk nodes**, and **community summaries**.
- The gain is simple: retrieval is no longer "find text that sounds similar." It becomes "find the *relevant subgraph*."



Once facts are stored as a graph, retrieval can follow links rather than isolated paragraphs.

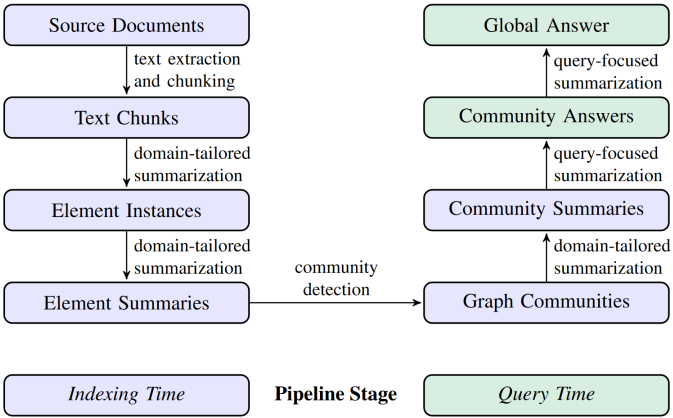
Graph-Based RAG: Offline vs. Online

Offline indexing

- chunk the corpus
- extract entities and relations
- build the graph and any auxiliary indexes
- optionally detect communities and summarize them

Online querying

- interpret or rewrite the question
- retrieve nodes, edges, chunks, or communities
- expand to a relevant subgraph
- build the prompt and ask the LLM to answer



GraphRAG-style intuition: communities are built offline, then query-focused summaries are composed online.

Knowledge Graph Triples: Examples Across Research Corpora

Once relationships are explicit, retrieval can follow links, communities, paths, and neighborhoods — not only semantic distance.

Corpus	Example triples
FOMC archive	<June 2024 minutes, discusses, inflation expectations> <Powell press conference, mentions, labor-market rebalancing>
NBER / RePEc papers	<Paper A, estimates, Phillips curve slope> <Paper A, uses data, CPS> <Paper B, cites, Paper A>
SEC filings	<Bank X 10-K, reports risk, deposit beta> <Bank X, exposed to, commercial real estate>
Central bank speeches	<ECB speech 2022-09, frames, transitory inflation> <Lagarde, cited by, FOMC speakers>

Why it matters for economists: Questions like *"Which Phillips-curve papers use survey expectations and are cited by post-2020 FOMC speeches?"* are *graph queries*, not semantic-similarity queries.

Vanilla RAG vs. GraphRAG: A Direct Comparison

Question type	Vector / hybrid RAG	GraphRAG
Specific fact lookup	Strong when the exact chunk is nearby	Strong if fact is attached to an entity or path
Relationship query	Often requires several retrieved chunks and luck	Natural: traverse edges and retrieve neighborhoods
Global summarization	Top-K chunks under-cover the corpus	Community reports summarize modules of the corpus
High redundancy corpus	May retrieve near-duplicate boilerplate	Can collapse repeated facts into entities/communities
Cost and complexity	Simpler, cheaper, easier to debug	More expensive offline; KG quality matters

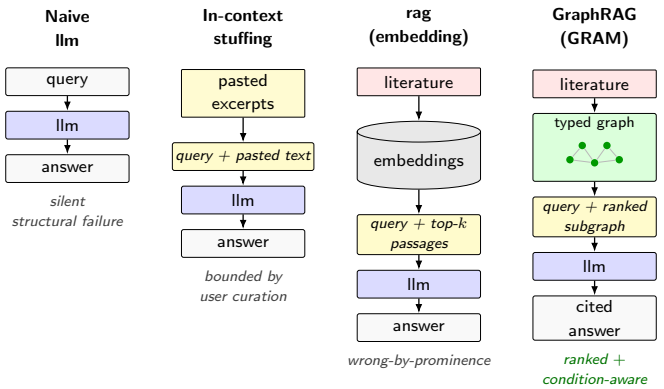
Research rule: start with vector + BM25 + re-rank. Move to graph or hierarchical retrieval *only* when the question requires relationships or corpus-level themes you can't get from local similarity.

Case Study: GraphRAG over the OLG Economics Literature (GRAM)

- A research-grade GraphRAG — the instructor's own **GRAM** project — built over a curated **157-paper overlapping-generations (OLG)** macro corpus. Think of it as the *research-grade escalation* of the small offline OLG demo coming in Topic 8.4b, not a replacement.
- **One running query** (we trace it through four mechanisms next):
  "An OLG life-cycle model with idiosyncratic and aggregate shocks and incomplete intergenerational asset markets — which solver class is admissible for the joint stationary distribution in recursive equilibrium?"
- **Why economics is special:** every paper is **math** (the analytical scaffold) *and* **prose** (the mechanism). *Math is analysis; prose is mechanism.*
  - Math-IR systems retrieve theorems/equations; legal/clinical RAG retrieves prose. An econ assistant needs *both*.
  - GRAM sits between those worlds: a domain-typed graph over economic models.

Four Context Mechanisms on One Query

"OLG life-cycle, idiosyncratic + aggregate shocks, incomplete intergenerational asset markets — which solver class is admissible for the joint stationary distribution in recursive equilibrium?"



Wrong-by-Prominence vs. Condition-Aware Retrieval

- **Wrong-by-prominence:** pure embedding retrieval routes the query to the *modal* (most-prominent) paper, regardless of whether it fits the asked-for conditions:
  - "OLG with capital" ⇒ Diamond (1965), *not* Auerbach–Kotlikoff (1987).
  - "heterogeneous-agent macro" ⇒ Aiyagari, *not* Krusell–Smith.

  The vocabulary is present; the applicability filter is absent.

- **Condition-aware:** admissibility is *conditional* — change one model assumption (e.g. add aggregate risk) and a different solver class becomes admissible. Embedding similarity cannot encode that dependency; a walk over a *typed* graph of model objects can.

Honest framing. "Wrong-by-prominence" and "condition-aware" name each mechanism's *failure mode* on this query type. GRAM's condition-aware advantage over vanilla RAG is the **gap a planned three-condition evaluation (naive / RAG / GraphRAG) is designed to measure** — a design goal and hypothesis, not yet a reported win.

Elements, Not Chunks: A Typed Retrieval Unit

- Standard RAG (Topic 8.2) embeds a ∼**512-token text chunk**. GRAM's retrieval unit is a typed **element**.
- **The PromptEcon schema:** 3 phases (Environment → Equilibrium & solution → Quantification), **9 categories**, **51 elements**, **7 typed edges** (`enters_objective`, `enters_constraint`, `clears_in`, `derives`, ...).
- Each paper → one provenance-carrying JSON of element records (`canonical_name`, prose description with inline TEX, `source_lines`). **One embedding per element**, not per chunk; cross-paper merge happens at `(element, canonical_name)`, not paper-level similarity.
- **Why it matters:** a chunk returns surface phrases ("aggregate shocks," "recursive equilibrium") without saying *what enters which constraint* or *what clears in which market*. Typed elements + edges preserve exactly that structure.

Slogan: *the schema is the research; extraction is the mechanical pass.* The 9×51 vocabulary is human-curated from five textbook traditions; per-paper instantiation is automated.

Corpus Construction as Engineering (Not Bibliography)

"A retrieval system is only as good as the documents in its index." GRAM treats corpus building as engineering with input contracts, artifacts, and acceptance tests.

- **157 papers**, three tiers (Foundations 12 / pre-2010 core 55 / published 2010+ 43 / frontier WPs 47); a 4-attribute model-class *inclusion gate* fixes each paper's role.
- **Three design constraints worth copying:**
  1. *Coverage vs. focus* — index the full mechanism vocabulary, not the curator's reading list.
  2. *Post-cutoff share* — a non-trivial fraction (here 34 entries dated >2024) must post-date the LLM's training cutoff, so RAG can *demonstrably* add value.
  3. *Citation connectivity* — documents must actually cite each other, or graph-traversal retrieval cannot reach them.
- Reproducible by construction: every keep/drop decision logged; an 11-criterion acceptance audit (passed 9/11); MinerU for layout-aware PDF→Markdown (math preserved); Qwen3-Embedding-8B (∼$0.25 to vectorize the pilot corpus).

Caveats. GRAM is *one* research pipeline (the instructor's), shown beside Microsoft GraphRAG / LightRAG / HippoRAG / RAPTOR above — not a canonical standard. It is unpublished, in-

Choosing the Retrieval Architecture

Question type	Best first choice	Why
Precise factual lookup or citation hunt	Hybrid dense + sparse RAG	Exact strings, dates, and citations matter more than global structure.
Specific multi-hop entity question	Graph-based RAG	The answer depends on linked entities, paths, or neighborhoods rather than one chunk.
Broad abstract summarization over a large corpus	GraphRAG / RAPTOR-style hierarchy	You need coverage of themes or communities, not just the closest paragraphs.
Fast prototype on a changing corpus	Vanilla RAG first	A flat vector index is easy to build and update; add graph structure only if it buys real accuracy.

Rule of thumb: start with hybrid dense+sparse RAG. Escalate to graph or hierarchical retrieval when the task requires *relationships*, *coverage*, or *multi-hop reasoning*.

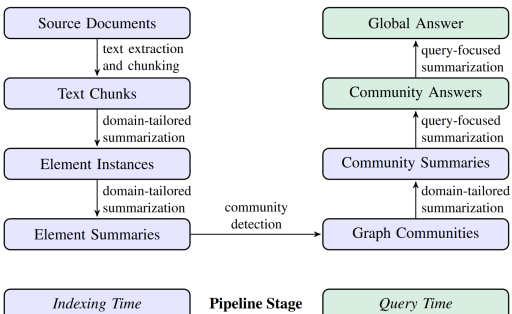
Representative Systems You Should Recognize

System	Core idea	Best at	What to remember
Microsoft GraphRAG	Build a KG, detect communities, summarize them, then retrieve by community plus local evidence	Query-focused summarization	Strong when the user asks for an overview of a large corpus rather than one citation.
LightRAG	Dual-level retrieval: local entity-level evidence plus high-level thematic evidence	Mixed local/global questions	A pragmatic design that tries to keep graph retrieval lightweight.
HippoRAG	Expand a memory graph with KNN links, then run personalized PageRank from the query	Multi-hop recall	Explicitly treats retrieval as traversing associative memory.
RAPTOR	Recursively cluster chunks and summarize the clusters into a tree	Long-document and abstract QA	Not a full KG, but the same core idea: retrieve from higher-level structure when chunks alone are too local.

Key references: Edge et al. (2024), Guo et al. (2024), Gutierrez et al. (2024), and Sarthi et al. (2024).

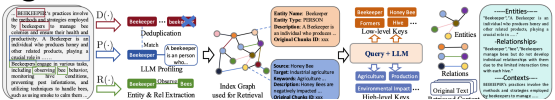
Visual Snapshots: GraphRAG and LightRAG

GraphRAG



Community summaries provide global coverage for abstract questions.

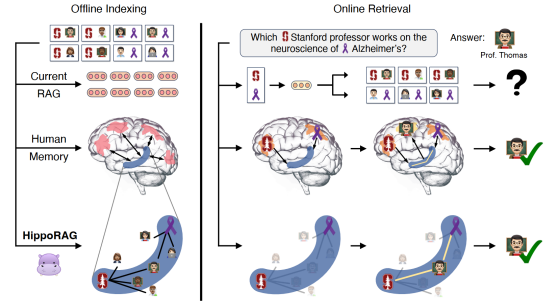
LightRAG



Dual-level retrieval: local entities plus higher-level themes.

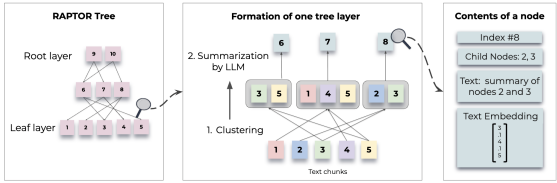
Visual Snapshots: HippoRAG and RAPTOR

HippoRAG



Personalized PageRank expands a memory-like graph neighborhood.

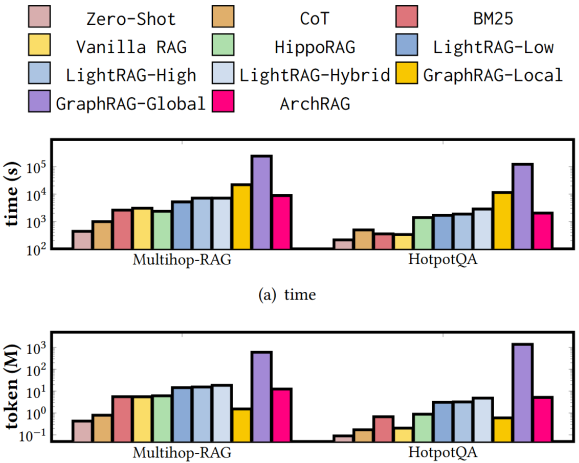
RAPTOR



Recursive clustering and summarization into a tree.

What Recent Graph-RAG Benchmarks Suggest

- **Chunk quality still dominates.** Better chunks help almost every architecture.
- **High-level structure helps abstract questions.** Community reports and hierarchies beat a bag of similar chunks when the user wants synthesis.
- **Specific QA can still need structure.** Hard factual questions often require linking several local pieces of evidence.
- **Graph maintenance is expensive.** Build cost, token cost, and dynamic updates remain real bottlenecks.



Illustrative benchmark from the donor deck: graph-RAG variants can buy capability, but often at sharply different token and latency costs.

Future Directions: Graph, Multimodal, Agentic

Better graphs

- Higher-quality entity and relation extraction
- Attribute-rich nodes and edges
- Efficient incremental updates as new docs arrive

Richer evidence

- Tables, figures, equations, and PDFs
- Cross-modal alignment of text and data
- Provenance-preserving retrieval

Agentic RAG

- Agent chooses vector vs. graph vs. web
- Query rewriting and retrieval loops
- Automatic parameter tuning and evals

Macro research anchor: A serious literature- or policy-text assistant will eventually need all three — graph structure for relationships, multimodal parsing for papers and reports, and an agent loop for difficult questions.

Next (Topic 8.4a): augmented generation — how the retrieved evidence becomes a cited, refusable answer, and five economic-applications walkthroughs. **Topic 8.4b** then runs the OLG demo and bridges to *Lecture 9*'s agentic loop.